Cost Breakdown

GA Prompt Optimiser for LLM Code Generation

Complete API Cost Analysis with Official Pricing

Group 1

Computational Intelligence — Spring 2026

April 17, 2026

All prices sourced from the official Anthropic API pricing page [1].
Token measurements taken from the actual APPS dataset evolution pool.

Contents

1 Pricing Sources **2**
- 1.1 Model Pricing (Standard API) . . . 2
- 1.2 Batch API Pricing (50% Discount) . . . 2

2 Token Measurements **3**

3 API Call Counts **4**
- 3.1 Derivation . . . 4

4 Full Cost Breakdown **5**
- 4.1 Token Totals . . . 5
- 4.2 Cost by Component . . . 5
- 4.3 Cost per Generation . . . 5
- 4.4 Summary Table . . . 6

5 Optimisation Options **7**
- 5.1 Three Levers to Reduce Cost . . . 7
- 5.2 Optimised Configurations . . . 7
- 5.3 Batch API Impact . . . 7

6 Runtime Cost Tracking **8**

Chapter 1

Pricing Sources

All prices in this document are sourced from the official Anthropic API pricing page, accessed April 2026 [1].

1.1 Model Pricing (Standard API)

Model	Input ($/MTok)	Output ($/MTok)
Claude Haiku 4.5 (`claude-haiku-4-5-20251001`)	1.00	5.00
Claude Sonnet 4.6 (`claude-sonnet-4-6`)	3.00	15.00
Claude Opus 4.6 (`claude-opus-4-6`)	5.00	25.00

Table 1.1: Standard API pricing. MTok = million tokens. Source: [1].

1.2 Batch API Pricing (50% Discount)

Model	Input ($/MTok)	Output ($/MTok)
Claude Haiku 4.5 (Batch)	0.50	2.50
Claude Sonnet 4.6 (Batch)	1.50	7.50

Table 1.2: Batch API pricing (50% discount, asynchronous). Source: [1].

Why two models?

Haiku 4.5 generates code (cheap, deterministic at temperature 0) — this is the model being optimised. **Sonnet 4.6** performs crossover and mutation (needs creativity, temperature 0.7) — this is the evolutionary operator. 97% of the cost comes from Haiku calls; Sonnet is used sparingly.

Chapter 2

Token Measurements

Token sizes were **measured from real APPS data**, not estimated. The evolution pool of 100 problems was loaded and measured; solution files from the dataset were used to gauge output length.

Call type	Input (tokens)	Output (tokens)	How measured
Fitness (code gen)	280	138	system prompt ∼48 + question ∼232
Crossover	350	100	meta-prompt ∼150 + two parents ∼100 each
Mutation (any type)	200	100	meta-prompt ∼100 + one parent ∼100

Table 2.1: Measured token sizes per API call. Source: real APPS evolution pool.

Measurement methodology

- **Question length**: Average of `len(question)` for 100 sampled APPS problems, divided by 4 (chars-to-tokens approximation). Result: 876 chars ≈ 219 tokens. Plus system prompt (∼48 tokens) and function-name suffix (∼13 tokens) = 280 total input.
- **Output length**: Average of `len(solution)` from 30 sampled `solutions.json` files, divided by 4. Result: 552 chars ≈ 138 tokens.
- **Question range**: 164 – 5,400 chars (41 – 1,350 tokens). The 280-token average accounts for the full distribution.

Chapter 3

API Call Counts

3.1 Derivation

Phase	**Model**	**Calls**	**Formula**
Gen-0 fitness	Haiku 4.5	1500	$15pop \times 100problems$
Gen 1–20 fitness	Haiku 4.5	26000	$13offspring \times 20gen \times 100$
Holdout evaluation	Haiku 4.5	200	$1bestprompt \times 200problems$
Crossover	Sonnet 4.6	208	$13 \times 20 \times 0.80$
Mutation: inject	Sonnet 4.6	65	260×0.25
Mutation: delete	Sonnet 4.6	39	260×0.15
Mutation: rephrase	Sonnet 4.6	26	260×0.10
Total Haiku		**27700**	
Total Sonnet		**338**	
Grand Total		**28038**	

Table 3.1: API call counts by phase.

Key assumptions

- Population size = 15, elitism = 2 $\Rightarrow$ 13 new offspring per generation.
- Total offspring across 20 generations = $13 \times 20 = 260$.
- Crossover rate = 0.80, so $\sim$208 crossover calls.
- Mutation probabilities are independent per offspring: inject 0.25, delete 0.15, rephrase 0.10.
- Elite individuals are *not* re-evaluated (fitness carried from previous generation).

Chapter 4

Full Cost Breakdown

4.1 Token Totals

Model	Input tokens	Output tokens	Total tokens
Haiku 4.5	7756000	3822600	11578600
Sonnet 4.6	98800	33800	132600
Combined	**7854800**	**3856400**	**11711200**

Table 4.1: Token totals. Haiku: $27700 \times 280/27700 \times 138$. Sonnet: $208 \times 350 + 130 \times 200$ / $(208 + 130) \times 100$.

4.2 Cost by Component

Component	Input cost	Output cost	Total	Share
Haiku 4.5 (fitness + holdout)	7.76	19.11	26.87	97.1%
Sonnet 4.6 (operators)	0.30	0.51	0.80	2.9%
Grand Total	**8.05**	**19.62**	**27.67**	100%

Table 4.2: Cost breakdown by model. All prices in USD.

Where the money goes

97% of the cost is Haiku fitness evaluation (code generation). Output tokens dominate because Haiku output ($5/MTok) is 5× more expensive than input ($1/MTok), and code generation produces substantial output. The Sonnet operators (crossover + mutation) are negligible at $0.80 total despite Sonnet being 3× more expensive per token.

4.3 Cost per Generation

Component	Cost per generation
Fitness (13 offspring × 100 problems)	1.26
Operators (crossover + mutations)	0.04
Total per generation	**1.30**

Table 4.3: Cost per generation (gen 1–20). Gen 0 costs $1.46 (all 15 seeds evaluated).

4.4 Summary Table

Metric	**Value**
Total API calls	28038
Total input tokens	7854800
Total output tokens	3856400
Total input cost	$8.05
Total output cost	$19.62
Grand total cost	**$27.67**
Estimated wall-clock time	∼15.4 hours
Cost per generation	$1.30

Table 4.4: Grand summary — full 20-generation run.

Chapter 5

Optimisation Options

5.1 Three Levers to Reduce Cost

1. **Reduce evolution pool size** (100 → 50 or 75 problems): Halves or reduces fitness calls proportionally. GAAPO [2] showed that bandit-based selection with 50 samples matched full evaluation quality.

2. **Reduce generations** (20 → 10 or 15): GAAPO found that larger populations with fewer generations outperformed smaller populations with more generations [2].

3. **Batch API** (50% off Haiku): Processes calls asynchronously. Fitness evaluation is embarrassingly parallel — all 100 problems per individual are independent.

5.2 Optimised Configurations

Config	Pop	Gen	EvoPool	Cost	Hours	Savings
Full run (current)	15	20	100	27.67	15.4	—
Option A (cheapest)	10	15	50	6.77	3.7	75%
Option B (balanced)	12	15	75	12.39	6.8	55%
Option C (half gens)	15	10	100	14.66	8.2	47%
Option D (half pool)	15	20	50	14.24	7.7	49%

Table 5.1: Optimised configurations with cost and time estimates.

Recommended: Option B ($12.39, 6.8 hours)

Pop=12, Gen=15, EvoPool=75, Holdout=150. Fits within a $15 budget and 10 hours of wall-clock time. Can be run twice with different seeds for $25 total and 13.6 hours, providing reproducibility. Changes only 4 lines in `config.py`.

5.3 Batch API Impact

Configuration	Standard API	With Batch API
Full run (Pop=15, Gen=20, Pool=100)	27.67	14.24
Option B (Pop=12, Gen=15, Pool=75)	12.39	6.39

Table 5.2: Cost with Batch API (50% off Haiku). Sonnet operators are unchanged.

Chapter 6

Runtime Cost Tracking

The `cost_tracker.py` module (see Code Explanation document) automatically logs every API call's exact token count and computes the real cost. After each run, a JSON file is saved to `Costs/cost_<timestamp>.json` with this structure:

```
{
  "grand_total_usd": 27.67,
  "breakdown_by_phase": {
    "fitness":        { "calls": 27700, "total_cost_usd": 26.87 },
    "crossover":      { "calls": 208,   "total_cost_usd": 0.53 },
    "mutate_inject":  { "calls": 65,    "total_cost_usd": 0.14 },
    "mutate_delete":  { "calls": 39,    "total_cost_usd": 0.08 },
    "mutate_rephrase":{ "calls": 26,    "total_cost_usd": 0.06 }
  },
  "breakdown_by_model": {
    "claude-haiku-4-5-20251001": { "total_cost_usd": 26.87 },
    "claude-sonnet-4-6":         { "total_cost_usd": 0.80 }
  }
}
```

Each run creates a **new file** — they never overwrite. This gives a full cost history across all experiments. The `grand_total_usd` figure in each file is the *actual measured cost* (from real API token counts), not an estimate.

Bibliography

[1] Anthropic, "Pricing — Claude API Docs," https://docs.anthropic.com/en/docs/about-claude/pricing, accessed April 2026.

[2] X. Sécheresse, J.-Y. Guilbert-Ly, and A. Villedieu de Torcy, "GAAPO: Genetic algorithmic applied to prompt optimization," *Frontiers in Artificial Intelligence*, vol. 8, Art. no. 1613007, Sep. 2025, https://doi.org/10.3389/frai.2025.1613007.

[3] Q. Guo *et al.*, "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers," in *Proc. ICLR*, 2024.

[4] D. Hendrycks *et al.*, "Measuring coding challenge competence with APPS," *NeurIPS Datasets and Benchmarks*, 2021, https://arxiv.org/abs/2105.09938.